

Setting Up: conda, Jupyter and Google Colab

Contents

1. Overview	2
2. Anaconda, Miniforge and conda	2
2.1 What Anaconda and conda are	3
2.2 Miniforge: the installer we use	3
2.3 Installing Miniforge	3
2.4 Navigator and Spyder	4
3. Jupyter notebooks	5
3.1 Starting Jupyter	5
3.2 Code cells and Markdown cells	5
3.3 The kernel	6
3.4 Loading a dataset	6
3.5 Saving and sharing a notebook	7
4. Virtual environments	7
4.1 Why one environment per project	7
4.2 Creating and using an environment	8
4.3 Removing an environment	9
5. The environment used in these Notes	10
5.1 Recreating the environment from a file	10
5.2 Key versions	10
5.3 Two fixes for the profiling library	11
6. Kaggle notebooks	11
7. Google Colab	12
7.1 Data on Colab	13
7.2 Libraries on Colab	13
8. Kaggle datasets on Colab	14
9. Choosing where to work	15
10. Summary	16
11. Key terms	16

1. Overview

Key point: We can write ML code on our own computer (conda plus Jupyter) or in the browser (Kaggle or Google Colab); both give us the same kind of notebook.

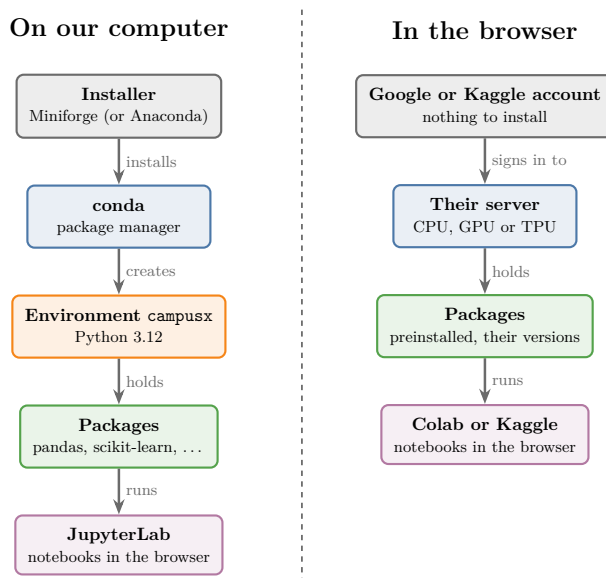


Figure 1: Two ways to get a working ML setup

ML work needs Python plus many libraries: pandas for tables, NumPy for numbers, scikit-learn for models, and plotting libraries. Installing and matching all of them by hand is slow and error-prone. Figure 1 shows the two ways around this:

- **On our computer:** an installer gives us **conda**, conda builds a separate box of Python and libraries, and Jupyter runs notebooks from that box.
- **In the browser:** Kaggle or Google Colab give us a ready-made notebook on their servers. Nothing to install, but the library versions are theirs.

Every command in this Note matches the environment the rest of these Notes are built with. It is pinned in the file `environment.yml` (Section 5). The Notebook for this Note (`notebook.ipynb`) checks a setup: it prints which Python and which library versions are really in use.

2. Anaconda, Miniforge and conda

Key point: conda installs Python and data science libraries in one place; Miniforge is the small installer for it that we use, and Anaconda is the large one.

2.1 What Anaconda and conda are

Key point: A distribution bundles Python with hundreds of libraries; conda is the tool inside it that installs and updates them.

Data science uses many libraries, and each depends on others. Installing them one by one, with versions that work together, quickly becomes hard. A **distribution** solves this: one installer that sets up Python together with the libraries we need.

Anaconda is the best-known data science distribution. It installs Python, about 250 popular libraries, and tools such as Jupyter. At its heart is **conda**, a **package manager**: a program that downloads libraries (called **packages**), works out which versions fit together, and installs them.

Extra: pip vs conda. Python also has its own package manager, **pip**, which downloads packages from **PyPI** (the Python Package Index). conda can also install things that are not Python, such as Python itself and the compiled maths libraries under NumPy. We use conda for almost everything and pip only for packages that conda does not have.

2.2 Miniforge: the installer we use

Key point: Miniforge installs only conda and Python, and gets packages from the free conda-forge channel; we add the rest ourselves.

A **channel** is an online store of conda packages. Anaconda's own channel is the default in the Anaconda installer. **conda-forge** is a community channel with more packages, often newer ones.

Miniforge is a small installer (under 100 MB) that sets up only conda, Python and a few basics, with conda-forge as its only channel. Everything else we install into our own environments (Section 4). This is the setup these Notes use.

The full Anaconda distribution works too: every **conda** command in this Note is the same. It is a much bigger download (about 1 GB today) and installs hundreds of libraries we may never use.

Extra: Why Miniforge rather than Anaconda? Anaconda's terms require a paid licence for its default channel in organisations with 200 or more people. conda-forge is free for everyone. Miniforge also starts small, so the only libraries on our machine are the ones our projects need.

2.3 Installing Miniforge

Key point: Download the installer for our system from the Miniforge GitHub page, run it, and accept the defaults.

The installers are on the Miniforge release page (github.com/conda-forge/miniforge). On Linux and macOS, two commands in a terminal download and run the right one:

Python: Installing Miniforge on Linux or macOS (terminal commands, not Python).

```
URL=https://github.com/conda-forge/miniforge/releases/latest/download
```

```
curl -L -O "$URL/Miniforge3-$(uname)-$(uname -m).sh"
```

```
bash Miniforge3-$(uname)-$(uname -m).sh
```

`$(uname)` fills in the system name (Linux or Darwin for macOS) and `$(uname -m)` the processor type (for example `x86_64`), so the URL points to the matching installer. The first line only stores the folder of the download page, `curl` downloads the installer, and `bash` runs it. We answer **yes** when it asks to initialise conda, so new terminals can find conda.

On Windows, we download `Miniforge3-Windows-x86_64.exe` from the same page and click through it with the default settings. Afterwards the Start menu has a **Miniforge Prompt**: a terminal where `conda` works. Every command in this Note is typed there on Windows.

To check the install, we open a new terminal and list the environments:

Python: Checking that conda works.

```
conda --version    # e.g. conda 24.11.3
```

```
conda env list     # every environment; "base" is always there
```

Extra: Older Anaconda guides show an “Individual Edition” download page and an installer of about 450 MB. Both have changed: the page now just says “Distribution”, and the installer is about twice that size. The steps stay the same: download, run, keep the defaults.

2.4 Navigator and Spyder

Key point: Anaconda adds a point-and-click control panel (Navigator) and a code editor (Spyder); Miniforge has neither, and we do not need them.

The full Anaconda distribution also installs:

- **Anaconda Navigator:** a window that lists environments and packages and starts tools with a click. It is a graphical front end for the same `conda` commands, and it can be slow.
- **Anaconda Prompt:** the terminal where `conda` works on Windows (Miniforge Prompt is the same thing).
- **Spyder:** a code editor for Python with a panel that shows the variables and tables in memory.

We use Jupyter (Section 3) instead of Spyder, and the terminal instead of Navigator. With Miniforge, `conda install spyder` adds Spyder to an environment if we want it.

3. Jupyter notebooks

Key point: A notebook mixes runnable code, its output and formatted text in one document, and runs one cell at a time.

3.1 Starting Jupyter

Key point: In a terminal with our environment active, `jupyter lab` starts a local server and opens it in the web browser.

Jupyter is the most popular tool for data science code. A **notebook** is a file (ending in `.ipynb`) made of **cells**, and each cell’s output appears right under it. **JupyterLab** is the program that opens and runs notebooks, inside a web browser.

Python: Starting JupyterLab.

```
conda activate campusx    # switch to our environment (Section 4)
cd path/to/our/project    # the folder we want to work in
jupyter lab               # starts Jupyter and opens the browser
```

The terminal keeps running while we work: it is the server behind the browser page. Closing it, or pressing **Ctrl+C** in it, stops Jupyter. The browser shows the files of the folder we started in, on the left.

Extra: “Jupyter Notebook” is also the name of an older, simpler interface, started with `jupyter notebook`. It is a separate package, **notebook**, which our environment does not include. JupyterLab opens the same `.ipynb` files, so nothing is lost. `conda install notebook` adds the classic interface if we prefer it.

Good practice is one folder per project. In the file browser, the folder button creates a new folder; we rename it (for example `100-days-of-ml`) and double-click to go inside. The **Python 3** button in the Launcher then creates a new, empty notebook there.

3.2 Code cells and Markdown cells

Key point: A code cell runs Python; a Markdown cell holds formatted text; **Shift+Enter** runs either one.

Every cell has a type, chosen from a menu at the top of the notebook:

- **Code cell:** holds Python. Running it shows the result underneath.
- **Markdown cell:** holds text written in **Markdown**, a simple way to format text with symbols. Running it turns the symbols into headings, bold text and lists.

Shift+Enter runs the current cell and moves to the next. Plain **Enter** only adds a new line inside the cell. Running cells one by one, and seeing each result straight away, is what makes notebooks so handy for exploring data.

Python: The Markdown we need most.

```
# Import the dataset      <- big heading (level 1)
## Load the CSV file     <- smaller heading (level 2)
Plain text becomes a paragraph.
**Two asterisks** make text bold.
<span style="color:red">HTML also works</span>
```

Because Markdown cells accept HTML, a notebook can also show images and videos. Many people publish notebooks as tutorials on GitHub and Kaggle, with the explanation in Markdown cells next to the code.

Extra: Notebooks can also hold interactive **widgets**: buttons, sliders and text boxes the reader can use. The `ipywidgets` package provides them; it is not in our environment. These Notes use Plotly and Dash for interactive parts instead.

3.3 The kernel

Key point: The kernel is the running Python behind a notebook; restarting it clears every variable.

Each open notebook is connected to a **kernel**: a Python process that runs the code cells and keeps the variables in memory. The **Kernel** menu can:

- **Interrupt** a cell that runs too long.
- **Restart** the kernel: all variables are lost, and we run the cells again from the top.
- **Shut down** the kernel when we are done, to free memory.

If Python crashes, or a package is changed underneath it, the notebook reports a **dead kernel**. Restarting the kernel fixes it.

3.4 Loading a dataset

Key point: Put the data file in the project folder (or upload it through the file browser), then read it with `pd.read_csv`.

Most datasets for learning come from **Kaggle** ([kaggle.com](https://www.kaggle.com)), a website of datasets and ML competitions; downloading needs a free account. A downloaded dataset usually arrives as a `.zip` file, which we extract on our computer.

To bring the file into the project folder, we either copy it there with the computer's file manager, or use the **upload** button (an upward arrow) in JupyterLab's file browser. Then pandas reads it into a table:

Python: Reading a CSV file that sits next to the notebook.

```
import pandas as pd

# the file name, as it appears in the file browser
```

```
df = pd.read_csv("aug_train.csv")
df.shape # (19158, 14): rows, columns
```

`aug_train.csv` is a Kaggle dataset of 19,158 people who took a data science course. `df` (short for `DataFrame`) is the usual name for a pandas table. The Note on working with CSV files covers `read_csv` in depth.

3.5 Saving and sharing a notebook

Key point: Notebooks save as `.ipynb` files, and can be exported as a Python script, a web page or a PDF.

JupyterLab saves the notebook as it is, as an `.ipynb` file. **File > Save and Export Notebook As** turns it into other formats:

Format	File	Good for
Notebook	<code>.ipynb</code>	Sharing with code, outputs and text; GitHub shows it as a page
Executable Script	<code>.py</code>	Running the code without Jupyter
HTML	<code>.html</code>	Reading in any browser, without Python
PDF	<code>.pdf</code>	Printing; needs LaTeX installed

Uploading the `.ipynb` file to GitHub is the usual way to share a project: GitHub draws the notebook, outputs included.

4. Virtual environments

Key point: An environment is a separate box of Python and packages; each project gets its own, so projects never break each other.

4.1 Why one environment per project

Key point: Working in the shared base environment mixes every project's packages, which makes projects hard to move to a server.

A **virtual environment** (in conda simply an **environment**) is a folder with its own copy of Python and its own packages. The installer creates one called **base**, which holds conda itself. With the full Anaconda distribution, base also holds all of Anaconda's libraries, and starting Jupyter from the Start menu uses base.

Working in base causes problems sooner or later:

- **Deployment:** to run a project on a server, the server needs the same packages. A project built in base does not record which packages it really used, so the server ends up with a long, heavy list.
- **Conflicts:** one project may need pandas 3 and an older one pandas 1.5. One shared box can hold only one version.
- **Breakage:** an upgrade made for one project can silently break another.

The fix is one fresh environment per project (Figure 2). It starts nearly empty, we install only what the project needs, and the server later gets exactly that list. It acts like a safe box for the project.

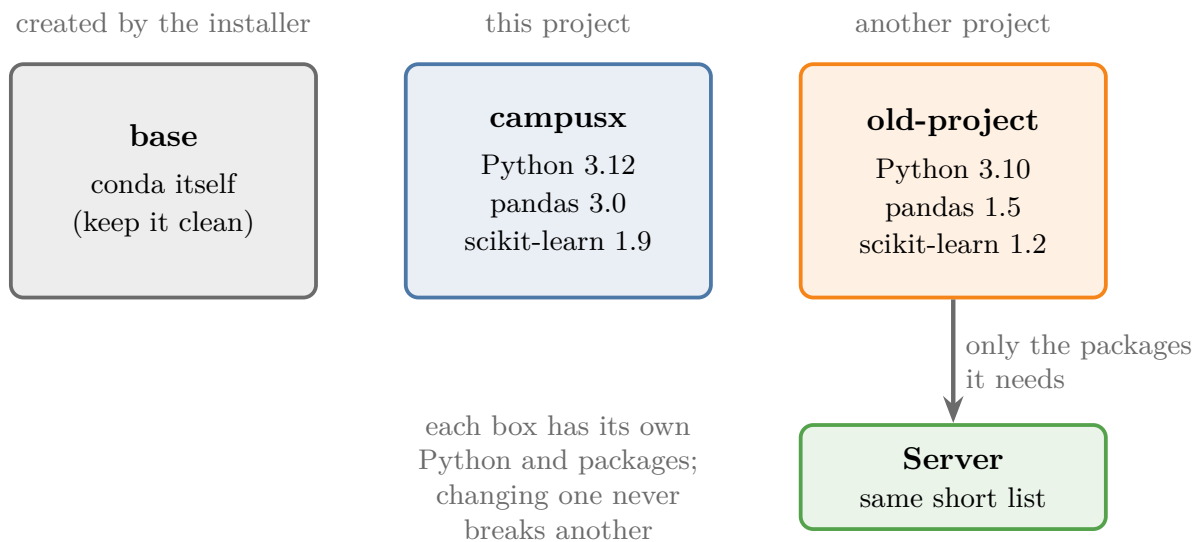


Figure 2: Separate environments, each with its own Python and packages

For a first ML project, working in one environment for everything is fine. Separate environments matter once projects are deployed or shared, and they are the standard practice.

4.2 Creating and using an environment

Key point: `conda create` makes the box, `conda activate` steps into it, and `conda install` fills it.

We create environments from a terminal (Miniforge Prompt or Anaconda Prompt on Windows). The prompt shows the active environment in brackets, for example `(base)`.

Python: The everyday conda commands.

```
# a new, nearly empty environment
conda create --name myproject python=3.12
# step into it: the prompt now starts with (myproject)
conda activate myproject
```

```
# install packages into it
conda install jupyterlab numpy pandas
# Jupyter now runs inside myproject
jupyter lab
# back to (base)
conda deactivate
```

conda asks for confirmation (Proceed ([y]/n)?) before it changes anything; we type y. Without python=3.12, the new environment would have no Python at all.

Jupyter is a package like any other. In a new environment it must be installed before `jupyter lab` works there. Once it runs from inside the environment, every notebook uses that environment's Python and packages.

A package can also be installed without leaving the notebook. Changing the packages under a running notebook can kill the kernel; after installing, we restart it.

Python: Installing from a notebook cell.

```
%pip install wordcloud
```

`%pip` installs into the environment that runs this notebook. The older form `!pip install wordcloud` sends the line to the terminal, and can install into a different Python by mistake. If the package is already there, pip answers `Requirement already satisfied`.

Anaconda Navigator can do the same with clicks: its **Environments** tab lists every environment and its installed packages, with buttons to create, install and remove.

4.3 Removing an environment

Key point: Deactivate the environment first, then remove it with `conda env remove`.

When a project is finished and saved (for example on GitHub), its environment can go:

Python: Removing an environment.

```
# never remove the environment we are in
conda deactivate
# delete it and all its packages
conda env remove --name myproject
# check: myproject is gone
conda env list
```

The older form `conda remove --name myproject --all` does the same.

If `conda activate myproject` still seems to work right after removing, the terminal was still inside the old environment. Closing the terminal and opening a new one shows the real state.

5. The environment used in these Notes

Key point: One file, `environment.yml`, lists every package and version; `conda env create` rebuilds the exact same environment from it.

5.1 Recreating the environment from a file

Key point: Two commands rebuild our `campusx` environment on any computer.

An **environment file** (`environment.yml`) lists an environment's name, its channel, and every package with its exact version. Pinning exact versions means everyone who builds from the file gets the same results as these Notes. Ours was made from the working environment with `conda env export --no-builds`.

The file has two parts: conda packages from conda-forge, and a `pip:` section for packages that conda-forge does not have. Recreating the environment takes one long download:

Python: Building the `campusx` environment (Linux or macOS terminal).

```
PIP_NO_DEPS=1 conda env create -f environment.yml
conda activate campusx
python -c "import sklearn; print(sklearn.__version__)" # 1.9.1
```

`-f` names the file. `PIP_NO_DEPS=1` tells the pip step to install exactly the listed versions and nothing else; Section 5.3 explains why. The last line checks that scikit-learn imports.

Extra: On Windows, the first line is split in two. In Miniforge Prompt: set `PIP_NO_DEPS=1`, then `conda env create -f environment.yml`. In PowerShell: `$env:PIP_NO_DEPS=1`, then the same command.

Figure 1's left side is now complete. From the project folder, `jupyter lab` opens notebooks that use exactly these versions.

5.2 Key versions

Key point: Python 3.12, pandas 3, NumPy 2 and scikit-learn 1.9 are the versions every Notebook is tested with.

The table lists the pinned versions that matter most for these Notes.

Package	Version	Used for
Python	3.12.14	The language
NumPy	2.5.3	Arrays and maths
pandas	3.0.6	Tables (DataFrames)
scikit-learn	1.9.1	ML models and preprocessing
SciPy	1.18.1	Statistics and scientific functions
statsmodels	0.15.0	Statistical models and tests
seaborn	0.13.2	Statistical charts
Plotly	7.1.0	Interactive charts

Package	Version	Used for
Dash	4.4.1	Small web apps from Python
Manim	0.20.1	Animated diagrams
JupyterLab	4.6.4	Running notebooks
fg-data-profiling	4.20.0	EDA reports (pip, see 5.3)

The file also holds the tools used to build these Notes, such as the speech-to-text library `faster-whisper` and the video downloader `yt-dlp`. They are not needed for the ML code.

5.3 Two fixes for the profiling library

Key point: The profiling library asks for pandas older than 3, so we install it without its dependency rules and switch pandas' text storage before using it.

The EDA report library from the Note on Pandas Profiling is installed with pip under the name `fg-data-profiling` (imported as `data_profiling`). Version 4.20 still declares that it needs pandas older than 3.0 and NumPy older than 2.6. A plain `pip install fg-data-profiling` would therefore quietly replace our pandas 3.0.6 with pandas 2.3.3.

To keep pandas 3, we install it with pip's `--no-deps` option. It means “install just this package, do not touch anything else”.

The helper packages it really needs (such as `phik`, `visions` and `wordcloud`) are listed in the file's `pip:` section with their own pins. This is why the build command in 5.1 sets `PIP_NO_DEPS=1`: it gives the whole pip step `--no-deps`.

Python: Adding the library to an existing environment by hand.

```
pip install --no-deps fg-data-profiling==4.20.0
```

The library then runs on pandas 3 with one setting. pandas 3 stores text columns in a new format (Arrow arrays) that the library cannot add up, and building a report fails with an error. One line before the report switches text back to plain Python strings:

Python: The setting used before every profiling report.

```
import pandas as pd
```

```
pd.set_option("mode.string_storage", "python")
```

The Notebook for the Pandas Profiling Note runs this line first. With pandas 2 it is not needed.

6. Kaggle notebooks

Key point: Kaggle runs notebooks in the browser right next to its datasets, so a dataset is one click away from code.

Instead of installing anything, we can work in the browser. **Kaggle notebooks** are Jupyter notebooks running on Kaggle's servers, with the usual libraries preinstalled. They are a good start for beginners, and for quick experiments on Kaggle datasets.

To start one, we open any dataset on Kaggle (for example the Titanic data) and click **New Notebook** (on the dataset's **Code** tab). Kaggle creates a notebook with that dataset already attached. It has the same code and Markdown cells and the same Shift+Enter.

The first cell Kaggle writes for us imports NumPy and pandas, and prints the path of every attached file. We copy a path from its output into `read_csv`:

```
Python: Reading an attached dataset on Kaggle.  
# path copied from the first cell's output  
df = pd.read_csv("/kaggle/input/titanic/train.csv")
```

Our own data works too. **Add Input** (the + in the right-hand panel) uploads a file from our computer as a new dataset; Kaggle asks for a unique name, processes it, and attaches it under `/kaggle/input/`.

Files our code writes go to `/kaggle/working/` and appear in the **Output** panel, from where we download them:

```
Python: Saving a table as a file to download.  
# appears under Output after a refresh  
df.to_csv("cars.csv", index=False)  
index=False leaves out the row numbers.
```

File > Download saves the notebook itself as `.ipynb`. **Save Version** saves it on Kaggle, and a public notebook can be read, copied and upvoted by others. Upvotes build a Kaggle profile, which some people show to employers.

Extra: Older guides say Kaggle notebooks have no GPU. That is no longer true: in the notebook settings, the **Accelerator** option offers GPUs and a TPU, with a limited number of free hours per week.

A **GPU** (graphics card) runs the big matrix maths of deep learning much faster than a normal processor (**CPU**). A **TPU** is Google's chip built only for that maths.

7. Google Colab

Key point: Colab is Google's version of a Jupyter notebook in the browser: notebooks save to Google Drive, and a free GPU is one setting away.

Google Colab (colab.research.google.com) runs Jupyter notebooks on Google's servers, using a Google account. It looks and works like the notebooks above: code and text cells, Shift+Enter, and **File > Download** as `.ipynb` or `.py`. Two things set it apart:

- **Google Drive:** every notebook is saved in our Drive automatically (in a folder called *Colab Notebooks*).

- **GPU and TPU: Runtime > Change runtime type > Hardware accelerator** switches the notebook to a GPU or TPU. Classic ML code (scikit-learn) does not use a GPU; deep learning code runs many times faster on one.

So we do not need an expensive computer to learn deep learning: Colab provides the hardware.

Extra: The free GPU has limits. A session ends after a while without activity and after at most about 12 hours. When it ends, the machine is wiped: variables and uploaded files are gone, only the notebook stays in Drive.

7.1 Data on Colab

Key point: Colab has no datasets of its own; we upload a file through the Files panel, or connect Google Drive.

The **Files** panel (the folder icon on the left) has an upload button. An uploaded file lands in `/content/`, the notebook's working folder, so `pd.read_csv("cars.csv")` finds it. Right-clicking it and choosing **Copy path** gives the full path.

Uploaded files disappear when the session ends. To keep data between sessions, we store it in Google Drive and connect Drive to the notebook:

Python: Connecting Google Drive in Colab.

```
from google.colab import drive

drive.mount("/content/drive")    # asks for permission once
df = pd.read_csv("/content/drive/MyDrive/data/cars.csv")
```

7.2 Libraries on Colab

Key point: Colab has the common libraries preinstalled, in Colab's own versions; `!pip install` adds anything missing.

pandas, NumPy, scikit-learn, Plotly, seaborn and many more are already installed on Colab. Their versions are chosen by Google and change every few months, so they may differ from the pins in Section 5.2. Running the Notebook for this Note on Colab lists the differences.

Python: Checking and adding libraries on Colab.

```
import pandas as pd
pd.__version__                # Colab's pandas version

!pip install fg-data-profiling # a library Colab does not have
```

Lines starting with `!` go to the machine's terminal. On Colab, `!pip install` and `%pip install` both install into the notebook's Python. After upgrading a library that was already imported, **Runtime > Restart session** loads the new version.

Most code in these Notes runs on Colab as it is. A few lines depend on new behaviour, such as pandas 3's text columns. If Colab's pandas is older, the `mode.string_storage` line from

Section 5.3 is simply not needed there.

8. Kaggle datasets on Colab

Key point: With a Kaggle API token, Colab downloads a Kaggle dataset straight from Kaggle’s servers, however big it is.

Colab has the GPU, Kaggle has the datasets. Downloading a large dataset (for example thousands of images) to our computer and uploading it to Colab again is slow. The Kaggle **API** (a way for programs to talk to a website) lets Colab fetch it directly, at the speed of Google’s network. Figure 3 shows the steps.

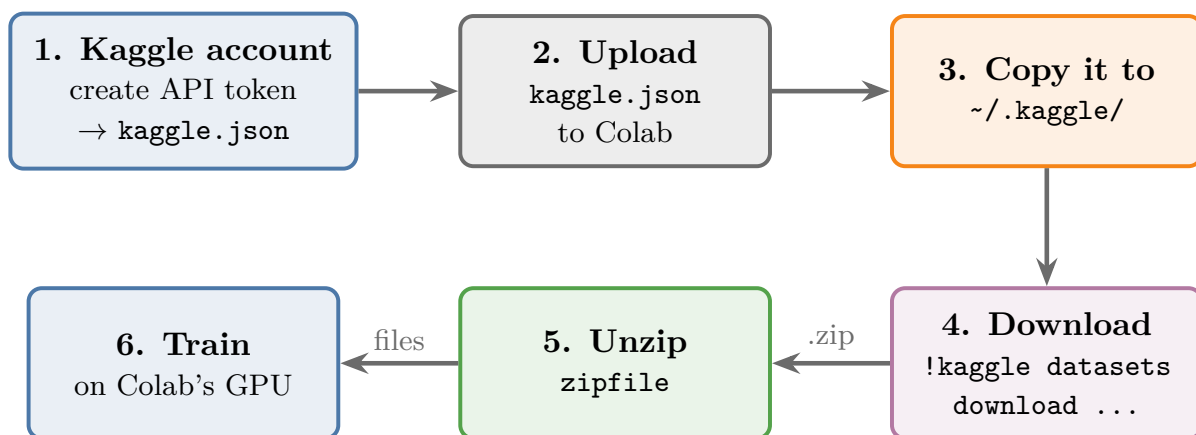


Figure 3: Downloading a Kaggle dataset straight into Colab

1. **Get a token.** On Kaggle, under the account settings, **Create New Token** downloads a file `kaggle.json`. It holds our username and a secret key.
2. **Upload it** to Colab through the Files panel.
3. **Copy it** to the folder where the Kaggle tool looks for it.
4. **Download the dataset.** On the dataset’s Kaggle page, the menu (the three dots) has **Copy API command**. We paste it into a cell with `!` in front.
5. **Unzip** the downloaded `.zip` file.
6. **Use the files** with the GPU.

Python: Steps 3 to 5 in Colab cells.

```
# Step 3: put the token where the kaggle tool expects it
!mkdir -p ~/.kaggle
!cp kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json

# Step 4: the command copied from the dataset's page
!kaggle datasets download -d owner/dataset-name
```

Step 5: unzip into a folder called data

```
import zipfile
with zipfile.ZipFile("dataset-name.zip") as z:
    z.extractall("data")
```

`mkdir -p` makes the hidden folder `.kaggle` in the home folder `~`. `chmod 600` makes the token file readable only by us; the Kaggle tool warns otherwise. `zipfile` is part of Python's standard library: `extractall` unpacks every file in the archive into the folder `data`.

Extra: `kaggle.json` is a password. It must never be shared, put in a public notebook or uploaded to GitHub. If it leaks, we delete the token on Kaggle and create a new one.

9. Choosing where to work

Key point: Kaggle and Colab for learning and quick experiments; a local conda environment for projects that will be deployed.

	Local (conda + JupyterLab)	Kaggle notebook	Google Colab
Install needed	Yes, once	No	No
Library versions	Ours, pinned	Kaggle's	Google's
Datasets	Download ourselves	Kaggle's, attached in one click	Upload, Drive, or Kaggle API
GPU	Only if our computer has one	Free weekly quota	Free, with session limits
Notebooks saved	On our disk	On Kaggle	In Google Drive
Best for	Projects and deployment	Kaggle data, sharing work	Deep learning on a GPU

For learning, Kaggle and Colab are enough, and many people use nothing else. A real project, one that goes to a server, needs a local environment with pinned versions.

10. Summary

Tool	What it is	Key command or click
Miniforge	Small installer: conda + Python, conda-forge channel	<code>bash Miniforge3-...sh</code>
Anaconda	Large installer with hundreds of libraries, Navigator, Spyder	Graphical installer
conda	Package and environment manager	<code>conda create</code> , <code>conda activate</code> , <code>conda install</code>
Environment file	Every package and version, pinned	<code>conda env create -f environment.yml</code>
JupyterLab	Runs notebooks in the browser	<code>jupyter lab</code>
Kaggle notebook	Notebook next to Kaggle's datasets	New Notebook on a dataset
Google Colab	Notebook on Google's servers, saved in Drive	Change runtime type for a GPU

- One environment per project; never fill up base.
- `conda activate campusx`, then `jupyter lab`, gives notebooks with exactly the versions in these Notes.
- Install fg-data-profiling with `--no-deps`, and set `mode.string_storage` to "python" before building a report.
- Colab and Kaggle have their own library versions; `!pip install` adds what is missing.
- A Kaggle API token lets Colab download Kaggle datasets directly. Keep it secret.

11. Key terms

Term	Meaning
Distribution	One installer that sets up Python together with many libraries
Anaconda	The best-known data science distribution, with Navigator and Spyder
conda	A package and environment manager for Python and other software
Package manager	A program that downloads and installs libraries in versions that fit together
Package	A library packed for installation
pip	Python's own package manager, which installs from PyPI
PyPI	The Python Package Index, the public store of Python packages

Term	Meaning
Channel	An online store of conda packages, such as conda-forge
conda-forge	A free, community-run conda channel
Miniforge	A small installer with only conda and Python, using conda-forge
Anaconda Navigator	Anaconda's point-and-click window for environments and packages
Spyder	A Python code editor that shows variables and tables in memory
Jupyter	Tool for notebooks that mix code, output and text
JupyterLab	The program that runs Jupyter notebooks in a web browser
Notebook	A <code>.ipynb</code> file of cells, each with its output underneath
Cell	One block of a notebook, holding either code or Markdown
Markdown	A simple way to format text with symbols such as <code>#</code> and <code>**</code>
Kernel	The running Python process behind a notebook
Virtual environment	A separate folder of Python and packages for one project
base environment	The environment the installer creates, holding conda itself
Environment file	A file (<code>environment.yml</code>) listing an environment's packages and versions
Kaggle	A website of datasets, ML competitions and browser notebooks
Google Colab	Google's browser-based Jupyter notebooks, saved in Google Drive
GPU	A graphics chip that runs deep learning maths much faster than a CPU
TPU	Google's chip built only for deep learning maths
API token	A secret file or key that lets a program use a website's API as us